

**LAPORAN UJIAN TENGAH SEMESTER
MATA KULIAH
PEMROGRAMAN BACKEND LANJUT PRAKTIKUM**



Disusun Oleh:

Fahmi Syihaab 434241120

**UNIVERSITAS AIRLANGGA
FAKULTAS VOKASI
TAHUN AJARAN 2026/2027**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Tujuan.....	5
1.3 Ruang Lingkup.....	6
BAB II	
ANALISIS DAN PERANCANGAN SISTEM.....	7
2.1 Kebutuhan Fungsional.....	7
2.2 Aktor Dan Hak Akses.....	8
2.3 Business Rule.....	9
2.4 Model Data.....	10
2.5 Perancangan API.....	11
BAB III	
IMPLEMENTASI.....	12
3.1 Migration.....	12
3.2 Seeder.....	21
3.3 Helper Response.....	26
3.4 Students.....	28
3.5 Courses.....	34
3.6 Enrollments.....	42
3.7 Route.....	46
BAB IV	
PENGUJIAN.....	49
1. POST /auth/login, Kredensial benar, 200, berisi access_token.....	49
2. POST /auth/login, Password salah, 401.....	49
3. POST /auth/login, Email tidak valid atau password kurang dari 8 karakter, 422.....	50
4. POST /auth/login, Gagal login lebih dari 5 kali dalam satu menit, 429.....	50
5. GET /auth/me, Token valid (admin dan mahasiswa), 200, mahasiswa menyertakan data students	
- Admin	
- Mahasiswa.....	51
6. GET /auth/me, Tanpa token atau token kedaluwarsa, 401.....	52
7. GET /students, Admin dengan page, per_page, filter, search, dan sort, 200 dengan data dan meta	
- page & per_page.....	52
8. GET /students, Login sebagai mahasiswa, 403.....	55
9. POST /students, Data valid, 201, user dan student terbentuk.....	55
10. POST /students, NIM atau email duplikat, 422.....	56
11. GET /students/{id}, Mahasiswa melihat data sendiri, 200 dengan total_sks dan batas_sks.....	56

12. GET /students/{id}, Mahasiswa melihat data mahasiswa lain, 403.....	57
13. GET /students/{id}, ID tidak ada atau sudah di-soft delete, 404 - Sebelum	
- Sesudah.....	58
14. PUT /students/{id}, Admin memperbarui data valid, 200.....	59
15. DELETE /students/{id}, Admin menghapus mahasiswa, 204, tidak muncul di daftar... 60	
16. GET /courses, Filter semester, search, dan available=true, 200 dengan terisi dan sisa_kuota.....	61
17. POST /enrollments, Mengambil mata kuliah yang memenuhi syarat, 201.....	61
18. POST /enrollments, Mengambil mata kuliah yang sama pada tahun akademik yang sama, 409.....	62
19. POST /enrollments, Mata kuliah berkuota penuh, 422.....	62
20. POST /enrollments, Total SKS melebihi batas (IPK $\geq$ 3,00; 2,50 sampai 2,99; < 2,50), 422 dengan pesan sisa SKS.....	63
21. POST /enrollments, Login sebagai admin, 403.....	63
22. DELETE /enrollments/{id}, Membatalkan enrollment milik sendiri, 204, sisa kuota bertambah.....	64
23. DELETE /enrollments/{id}, ID tidak ditemukan, 404.....	64
LAMPIRAN.....	65

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong institusi pendidikan tinggi untuk mendigitalkan layanan akademiknya. Salah satu layanan yang paling sering digunakan adalah Sistem Informasi Akademik (SIKAD), yang mengelola data mahasiswa, mata kuliah, dan Kartu Rencana Studi (KRS). Apabila proses tersebut masih dilakukan secara manual atau tanpa aturan yang ditegakkan oleh sistem, risiko kesalahan menjadi tinggi, misalnya mahasiswa mengambil mata kuliah yang sama dua kali, mengambil mata kuliah yang kuotanya sudah penuh, atau mengambil SKS melebihi batas yang diizinkan.

Untuk membangun layanan yang dapat digunakan oleh berbagai jenis aplikasi klien, seperti aplikasi web maupun aplikasi seluler, pengembangan sistem modern umumnya memisahkan sisi back end dari sisi front end. Pemisahan ini dijumpai oleh Application Programming Interface (API) yang mengikuti prinsip REST (Representational State Transfer). RESTful API memanfaatkan metode dan kode status HTTP serta format pertukaran data JSON, sehingga mudah dipahami, diuji, dan dikembangkan lebih lanjut.

Pengembangan API akademik memiliki beberapa tantangan teknis. Pertama, konsistensi data harus dijaga ketika banyak mahasiswa mengambil mata kuliah yang sama secara bersamaan, sehingga diperlukan transaction dan row locking agar kuota tidak terlampaui. Kedua, aspek keamanan harus diperhatikan, mencakup penyimpanan password dalam bentuk hash, autentikasi berbasis token, pembagian hak akses berdasarkan peran (admin dan mahasiswa), serta pembatasan agar mahasiswa hanya dapat mengakses KRS miliknya sendiri. Ketiga, data mahasiswa yang dihapus perlu tetap tersimpan untuk keperluan riwayat sehingga digunakan mekanisme soft delete.

Pada tugas ini, API dikembangkan menggunakan bahasa pemrograman Go dengan framework yang digunakan pada praktikum, serta PostgreSQL sebagai basis data. Go dipilih karena performanya yang baik, dukungan konkurensi yang kuat, dan penulisan kode yang ringkas, sedangkan PostgreSQL dipilih karena mendukung transaction, constraint, dan penguncian pada tingkat baris. Berdasarkan hal tersebut, dibangunlah RESTful API back end SIKAD Mini yang terdiri dari 10 endpoint dan menerapkan seluruh business rule yang ditetapkan.

1.2 Tujuan

Tujuan dari pengerjaan tugas ini adalah sebagai berikut.

1. Merancang basis data relasional SIKAD Mini yang terdiri dari tabel users, students, courses, dan enrollments beserta relasi dan constraint-nya.
2. Mengimplementasikan 10 endpoint RESTful API menggunakan Go dan PostgreSQL sesuai spesifikasi, dengan format response dan kode status HTTP yang seragam.

3. Menerapkan autentikasi berbasis token dan otorisasi berbasis peran (admin dan mahasiswa), disertai penyimpanan password dalam bentuk hash dan pembatasan percobaan login.
4. Menerapkan business rule pengambilan mata kuliah, yaitu batas SKS berdasarkan IPK, larangan pengambilan mata kuliah ganda pada tahun akademik yang sama, pengecekan kuota dengan row locking, dan pembatasan akses KRS hanya untuk pemiliknya.
5. Menyediakan migration dan seeder (minimal 1 admin, 20 mahasiswa, dan 10 mata kuliah) serta mengelola kode pada repository GitHub dengan riwayat commit bertahap.
6. Menguji seluruh endpoint untuk memastikan kesesuaian dengan spesifikasi dan business rule.

1.3 Ruang Lingkup

1. Sistem hanya mencakup sisi back end berupa RESTful API; antarmuka pengguna (front end) tidak dibuat.
2. Sistem memiliki dua peran pengguna, yaitu admin dan mahasiswa, serta tepat 10 endpoint sesuai definisi pada dokumen studi kasus.
3. Teknologi yang digunakan adalah Go, framework sesuai praktikum, dan PostgreSQL.
4. Data yang dikelola terbatas pada pengguna, mahasiswa, mata kuliah, dan KRS (enrollments). Fitur lain seperti nilai, jadwal kuliah, dosen, dan pembayaran tidak dicakup.
5. Pengelolaan data mata kuliah oleh admin dilakukan melalui seeder; endpoint untuk mata kuliah hanya berupa pembacaan daftar (GET /api/v1/courses).
6. Batas SKS dihitung per tahun akademik berdasarkan IPK terakhir yang tersimpan pada data mahasiswa, tanpa perhitungan IPK dari nilai.
7. Pengujian dilakukan melalui pengujian fungsional (Postman) dan unit test pada logika inti, dan tidak mencakup pengujian beban skala besar.

BAB II

ANALISIS DAN PERANCANGAN SISTEM

2.1 Kebutuhan Fungsional

Kebutuhan Fungsional	Aktor	Endpoint
Sistem dapat mengautentikasi pengguna dan mengembalikan access token	Publik	POST /api/v1/auth/login
Sistem dapat menampilkan profil pengguna yang sedang login	Admin, Mahasiswa	GET /api/v1/auth/me
Sistem dapat menampilkan daftar mahasiswa dengan pagination, filter, search, dan sort	Admin	GET /api/v1/students
Sistem dapat menambah mahasiswa sekaligus akun penggunanya	Admin	POST /api/v1/students
Sistem dapat menampilkan detail mahasiswa beserta mata kuliah yang diambil, total SKS, dan batas SKS	Admin, Mahasiswa (data sendiri)	GET /api/v1/students/{id}
Sistem dapat memperbarui data mahasiswa	Admin	PUT /api/v1/students/{id}
Sistem dapat menghapus mahasiswa secara soft delete	Admin	DELETE /api/v1/students/{id}

Sistem dapat menampilkan daftar mata kuliah beserta jumlah terisi dan sisa kuota	Admin, Mahasiswa	GET /api/v1/courses
Sistem dapat menambahkan mata kuliah ke KRS mahasiswa	Mahasiswa	POST /api/v1/enrollments
Sistem dapat membatalkan mata kuliah dari KRS milik sendiri	Mahasiswa	DELETE /api/v1/enrollments/{id}

2.2 Aktor Dan Hak Akses

Sistem memiliki dua aktor terautentikasi, yaitu admin yang mengelola data mahasiswa, dan mahasiswa yang melihat profil sendiri serta mengelola KRS miliknya. Pengguna yang belum login hanya dapat mengakses endpoint login.

No	Endpoint	Publik	Admin	Mahasiswa
1	POST /auth/login	Ya	-	-
2	GET /auth/me	-	Ya	Ya
3	GET /students	-	Ya	-
4	POST /students	-	Ya	-
5	GET /students/{id}	-	Ya	Ya (data sendiri)
6	PUT /students/{id}	-	Ya	-

7	DELETE /students/{id}	-	Ya	-
8	GET /courses	-	Ya	Ya
9	POST /enrollments	-	-	Ya
10	DELETE /enrollments/{id}	-	-	Ya (milik sendiri)

2.3 Business Rule

Aturan	Implementasi Rencana
Batas SKS per semester ditentukan oleh IPK terakhir	Fungsi MaxSKS(ipk) pada layer service
Mata kuliah yang sama tidak dapat diambil dua kali pada tahun akademik yang sama	Pengecekan di service dan UNIQUE constraint (student_id, course_id, tahun_akademik)
Mata kuliah dengan kuota penuh tidak dapat diambil	Hitung jumlah enrollments dengan row locking pada baris course
Mahasiswa hanya dapat mengakses dan mengubah KRS miliknya sendiri	Pencocokan student_id pada token dengan pemilik data

Batas SKS per semester :

IPK Terakhir	Maksimal SKS
IPK $\geq$ 3,00	24 SKS
2,50 sampai 2,99	21 SKS
IPK $<$ 2,50	18 SKS

2.4 Model Data

Tabel	Kolom Utama	Relasi
users	id, email, password, role (admin/mahasiswa)	1–1 ke students
students	id, user_id, nim (uniq), nama, prodi, angkatan, ipk_terakhir, deleted_at	1–N ke enrollments
courses	id, kode_mk (uniq), nama_mk, sks, semester, kuota	1–N ke enrollments
enrollments	id, student_id, course_id, tahun_akademik, created_at	uniq (student_id, course_id, tahun_akademik)

2.5 Perancangan API

No	Method	Endpoint	Akses	Sukses	Error
1	POST	/api/v1/auth/login	Publik	200	401, 422, 429
2	GET	/api/v1/auth/me	Semua role	200	401
3	GET	/api/v1/students	Admin	200	401, 403
4	POST	/api/v1/students	Admin	201	401, 403, 422
5	GET	/api/v1/students/{id}	Admin, Mahasiswa (sendiri)	200	401, 403, 404
6	PUT	/api/v1/students/{id}	Admin	200	401, 403, 404, 422
7	DELETE	/api/v1/students/{id}	Admin	204	401, 403, 404
8	GET	/api/v1/courses	Semua role	200	401
9	POST	/api/v1/enrollments	Mahasiswa	201	401, 403, 409, 422
10	DELETE	/api/v1/enrollments/{id}	Mahasiswa (milik sendiri)	204	401, 403, 404

BAB III

IMPLEMENTASI

3.1 Migration

```
=====
-- 1. ROLES & PERMISSIONS (RBAC)
=====

CREATE TABLE IF NOT EXISTS roles (

    name                VARCHAR(20)        PRIMARY KEY,

    description          VARCHAR(150)       NOT NULL,

    created_at           TIMESTAMPTZ        NOT NULL DEFAULT NOW()

);

INSERT INTO roles (name, description)

SELECT 'admin', 'Akses penuh terhadap seluruh data dan pengaturan'

WHERE NOT EXISTS (SELECT 1 FROM roles WHERE name = 'admin');

INSERT INTO roles (name, description)

SELECT 'mahasiswa', 'Akses mahasiswa untuk melihat data sendiri dan
mengelola KRS'

WHERE NOT EXISTS (SELECT 1 FROM roles WHERE name = 'mahasiswa');

INSERT INTO roles (name, description)

SELECT 'staff', 'Boleh melihat data seluruh user, tetapi tidak boleh
mengubah'

WHERE NOT EXISTS (SELECT 1 FROM roles WHERE name = 'staff');
```

```
CREATE TABLE IF NOT EXISTS permissions (  
    name          VARCHAR(50)          PRIMARY KEY,  
    description   VARCHAR(150)         NOT NULL  
);  
  
INSERT INTO permissions (name, description)  
  
SELECT 'student:list', 'Melihat daftar seluruh student'  
  
WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =  
'student:list');  
  
INSERT INTO permissions (name, description)  
  
SELECT 'student:read:any', 'Melihat data student mana pun'  
  
WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =  
'student:read:any');  
  
INSERT INTO permissions (name, description)  
  
SELECT 'student:read:self', 'Melihat data profil student sendiri'  
  
WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =  
'student:read:self');  
  
INSERT INTO permissions (name, description)  
  
SELECT 'student:create', 'Membuat data student baru'  
  
WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =  
'student:create');  
  
INSERT INTO permissions (name, description)  
  
SELECT 'student:update:any', 'Mengubah data student mana pun'  
  
WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =  
'student:update:any');  
  
INSERT INTO permissions (name, description)  
  
SELECT 'student:delete', 'Menghapus (soft delete) student'
```

```

WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =
'student:delete');

INSERT INTO permissions (name, description)

SELECT 'role:assign', 'Mengubah role milik pengguna lain'

WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =
'role:assign');

INSERT INTO permissions (name, description)

SELECT 'course:list', 'Melihat daftar mata kuliah'

WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =
'course:list');

INSERT INTO permissions (name, description)

SELECT 'enrollment:create', 'Mengambil KRS mata kuliah'

WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =
'enrollment:create');

INSERT INTO permissions (name, description)

SELECT 'enrollment:delete', 'Membatalkan KRS mata kuliah'

WHERE NOT EXISTS (SELECT 1 FROM permissions WHERE name =
'enrollment:delete');

CREATE TABLE IF NOT EXISTS role_permissions (

    role_name          VARCHAR(20)          NOT NULL REFERENCES roles(name) ON
DELETE CASCADE,

    permission_name    VARCHAR(50)          NOT NULL REFERENCES
permissions(name) ON DELETE CASCADE,

    PRIMARY KEY (role_name, permission_name)

);

-- Admin Permissions

```

```
INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'student:list' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'student:list');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'student:read:any' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'student:read:any');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'student:create' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'student:create');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'student:update:any' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'student:update:any');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'student:delete' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'student:delete');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'role:assign' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'role:assign');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'admin', 'course:list' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'admin' AND permission_name =
'course:list');

-- Staff Permissions

INSERT INTO role_permissions (role_name, permission_name)
```

```
SELECT 'staff', 'student:list' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'staff' AND permission_name =
'student:list');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'staff', 'student:read:any' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'staff' AND permission_name =
'student:read:any');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'staff', 'student:create' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'staff' AND permission_name =
'student:create');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'staff', 'course:list' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'staff' AND permission_name =
'course:list');

-- Mahasiswa Permissions

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'mahasiswa', 'student:read:self' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'mahasiswa' AND permission_name =
'student:read:self');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'mahasiswa', 'course:list' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'mahasiswa' AND permission_name =
'course:list');

INSERT INTO role_permissions (role_name, permission_name)

SELECT 'mahasiswa', 'enrollment:create' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'mahasiswa' AND permission_name =
'enrollment:create');

INSERT INTO role_permissions (role_name, permission_name)
```

```
SELECT 'mahasiswa', 'enrollment:delete' WHERE NOT EXISTS (SELECT 1 FROM
role_permissions WHERE role_name = 'mahasiswa' AND permission_name =
'enrollment:delete');
```

```
=====

-- 2. USERS

=====
```

```
CREATE TABLE IF NOT EXISTS users (

    id            SERIAL            PRIMARY KEY,

    email         VARCHAR(255)      NOT NULL UNIQUE,

    password      VARCHAR(255)      NOT NULL,

    role          VARCHAR(20)       NOT NULL DEFAULT 'mahasiswa' REFERENCES
roles(name) ON UPDATE CASCADE ON DELETE RESTRICT,

    created_at    TIMESTAMPTZ       NOT NULL DEFAULT NOW()

);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS users_email_unique_idx ON users
(LOWER(email));
```

```
CREATE INDEX IF NOT EXISTS users_role_idx ON users (role);
```

```
CREATE INDEX IF NOT EXISTS users_created_at_id_desc_idx ON users
(created_at DESC, id DESC);
```

```
=====

-- 3. REFRESH TOKEN (AUTH)

=====
```

```
CREATE TABLE IF NOT EXISTS refresh_token (

    id            BIGSERIAL         PRIMARY KEY,
```

```
        user_id      INTEGER          NOT NULL REFERENCES users(id) ON DELETE
CASCADE,

        token_hash   TEXT              NOT NULL UNIQUE,

        expires_at   TIMESTAMPTZ       NOT NULL,

        revoked_at   TIMESTAMPTZ,

        created_at   TIMESTAMPTZ       NOT NULL DEFAULT NOW()

);
```

```
CREATE INDEX IF NOT EXISTS refresh_token_user_id_idx ON refresh_token
(user_id);
```

```
=====

-- 4. STUDENTS

=====
```

```
CREATE TABLE IF NOT EXISTS students (

        id           SERIAL            PRIMARY KEY,

        user_id      INT               UNIQUE REFERENCES users(id) ON DELETE
CASCADE,

        nim          VARCHAR(12)       UNIQUE NOT NULL,

        nama         VARCHAR(100)      NOT NULL,

        prodi        VARCHAR(100)      NOT NULL,

        angkatan     INT               NOT NULL,

        ipk_terakhir NUMERIC(3,2)      NOT NULL DEFAULT 0.00 CHECK
(ipk_terakhir >= 0.00 AND ipk_terakhir <= 4.00),

        deleted_at   TIMESTAMPTZ       NULL,

        created_at   TIMESTAMPTZ       NOT NULL DEFAULT NOW()

);
```

```

CREATE INDEX IF NOT EXISTS students_prodi_idx ON students (prodi);

CREATE INDEX IF NOT EXISTS students_angkatan_idx ON students
(angkatan);

CREATE INDEX IF NOT EXISTS students_user_id_idx ON students (user_id);

CREATE INDEX IF NOT EXISTS students_deleted_at_idx ON students
(deleted_at);

CREATE INDEX IF NOT EXISTS students_created_at_id_desc_idx ON students
(created_at DESC, id DESC);

```

```

=====

-- 5. PRESTASI

```

```

=====

CREATE TABLE IF NOT EXISTS prestasi (

    id                SERIAL          PRIMARY KEY,

    student_id        INT            NOT NULL REFERENCES
students(id) ON DELETE CASCADE,

    name_prestation   VARCHAR(100)    NOT NULL,

    juara            VARCHAR(50)     NOT NULL

);

```

```

CREATE INDEX IF NOT EXISTS prestation_student_id_idx ON prestasi
(student_id);

```

```

=====

-- 6. COURSES (MATA KULIAH)

```

```

CREATE TABLE IF NOT EXISTS courses (

    id            SERIAL            PRIMARY KEY,

    kode_mk       VARCHAR(20)       UNIQUE NOT NULL,

    nama_mk       VARCHAR(100)      NOT NULL,

    sks           INT               NOT NULL CHECK (sks > 0),

    semester      INT               NOT NULL CHECK (semester > 0),

    kuota         INT               NOT NULL CHECK (kuota >= 0),

    created_at    TIMESTAMPTZ       NOT NULL DEFAULT NOW()

);

CREATE UNIQUE INDEX IF NOT EXISTS courses_kode_mk_key ON courses
(kode_mk);

```

```

=====

-- 7. ENROLLMENTS (KRS)

=====

```

```

CREATE TABLE IF NOT EXISTS enrollments (

    id            SERIAL            PRIMARY KEY,

    student_id    INT               NOT NULL REFERENCES students(id) ON
DELETE CASCADE,

    course_id     INT               NOT NULL REFERENCES courses(id) ON
DELETE CASCADE,

    tahun_akademik VARCHAR(30)      NOT NULL,

    created_at    TIMESTAMPTZ       NOT NULL DEFAULT NOW(),

    CONSTRAINT enrollments_student_course_tahun_unique UNIQUE
(student_id, course_id, tahun_akademik)

);

```

```

CREATE INDEX IF NOT EXISTS enrollments_course_id_idx ON enrollments
(course_id);

CREATE INDEX IF NOT EXISTS enrollments_student_id_idx ON enrollments
(student_id);

CREATE INDEX IF NOT EXISTS enrollments_tahun_akademik_idx ON
enrollments (tahun_akademik);

```

3.2 Seeder

```

// 1. Seed Admin

adminPassword := "admin12345"

hashedAdminPass, err :=
bcrypt.GenerateFromPassword([]byte(adminPassword), bcrypt.DefaultCost)

if err != nil {

    log.Fatalf("Gagal hash admin password: %v", err)

}

_, err = pool.Exec(ctx, `

    INSERT INTO users (email, password, role)

    VALUES ($1, $2, 'admin')

    ON CONFLICT (email) DO NOTHING

`, "admin@siakad.test", string(hashedAdminPass))

if err != nil {

    log.Fatalf("Gagal seed admin: %v", err)

}

fmt.Println("✓ Admin seeded")

```

```
// 2. Seed 20 Mahasiswa (Users + Students)

studentsData := []StudentSeed{

    {"mhs01@siakad.test", "202401000001", "Budi Santoso", "Teknik Informatika", 2024, 3.75},

    {"mhs02@siakad.test", "202401000002", "Siti Rahma", "Sistem Informasi", 2024, 3.40},

    {"mhs03@siakad.test", "202401000003", "Ahmad Fauzi", "Teknik Informatika", 2023, 2.85},

    {"mhs04@siakad.test", "202401000004", "Dewi Lestari", "Teknologi Informasi", 2023, 2.60},

    {"mhs05@siakad.test", "202401000005", "Rian Hidayat", "Teknik Informatika", 2022, 2.30},

    {"mhs06@siakad.test", "202401000006", "Maya Putri", "Sistem Informasi", 2022, 2.10},

    {"mhs07@siakad.test", "202401000007", "Eko Prasetyo", "Teknologi Informasi", 2024, 3.90},

    {"mhs08@siakad.test", "202401000008", "Fitriani", "Teknik Informatika", 2024, 3.20},

    {"mhs09@siakad.test", "202401000009", "Gita Gutawa", "Sistem Informasi", 2023, 2.95},

    {"mhs10@siakad.test", "202401000010", "Hendra Gunawan", "Teknologi Informasi", 2023, 2.70},

    {"mhs11@siakad.test", "202401000011", "Indah Permata", "Teknik Informatika", 2022, 2.45},

    {"mhs12@siakad.test", "202401000012", "Joko Widodo", "Sistem Informasi", 2022, 1.95},

    {"mhs13@siakad.test", "202401000013", "Kiki Amelia", "Teknologi Informasi", 2024, 3.55},

    {"mhs14@siakad.test", "202401000014", "Lukman Hakim", "Teknik Informatika", 2024, 3.10},
```

```

        {"mhs15@siakad.test", "202401000015", "Nia Ramadhani", "Sistem
Informasi", 2023, 2.80},

        {"mhs16@siakad.test", "202401000016", "Oscar Lawalata",
"Teknologi Informasi", 2023, 2.55},

        {"mhs17@siakad.test", "202401000017", "Putri Marino", "Teknik
Informatika", 2022, 2.25},

        {"mhs18@siakad.test", "202401000018", "Qory Sandioriva",
"Sistem Informasi", 2022, 3.85},

        {"mhs19@siakad.test", "202401000019", "Raffi Ahmad", "Teknologi
Informasi", 2024, 2.75},

        {"mhs20@siakad.test", "202401000020", "Sule Priakitiew",
"Teknik Informatika", 2024, 2.40},

    }

    for _, s := range studentsData {

        hashedNimPass, err :=
bcrypt.GenerateFromPassword([]byte(s.Nim), bcrypt.DefaultCost)

        if err != nil {

            log.Fatalf("Gagal hash password NIM %s: %v", s.Nim, err)

        }

        var userID int

        err = pool.QueryRow(ctx, `

            INSERT INTO users (email, password, role)

            VALUES ($1, $2, 'mahasiswa')

            ON CONFLICT (email) DO UPDATE SET email = EXCLUDED.email

            RETURNING id

        `, s.UserEmail, string(hashedNimPass)).Scan(&userID)

```

```

        if err != nil {

            log.Fatalf("Gagal insert user %s: %v", s.UserEmail, err)

        }

        _, err = pool.Exec(ctx, `

            INSERT INTO students (user_id, nim, nama, prodi, angkatan,
ipk_terakhir)

            VALUES ($1, $2, $3, $4, $5, $6)

            ON CONFLICT (nim) DO UPDATE SET

                nama = EXCLUDED.nama,

                prodi = EXCLUDED.prodi,

                angkatan = EXCLUDED.angkatan,

                ipk_terakhir = EXCLUDED.ipk_terakhir

        `, userID, s.Nim, s>Nama, s.Prodi, s.Angkatan, s.IpkTerakhir)

        if err != nil {

            log.Fatalf("Gagal insert student NIM %s: %v", s.Nim, err)

        }

    }

    fmt.Println("✓ 20 Mahasiswa seeded")

    // 3. Seed 10 Mata Kuliah

    coursesData := []CourseSeed{

        {"IF101", "Algoritma dan Pemrograman", 3, 1, 30},

        {"IF102", "Struktur Data", 4, 2, 25},

        {"IF103", "Pemrograman Web", 3, 3, 20},

```

```

{"IF104", "Basis Data", 3, 2, 35},

{"IF105", "Jaringan Komputer", 3, 4, 30},

{"IF106", "Kecerdasan Buatan", 3, 5, 15},

{"IF107", "Sistem Operasi", 3, 3, 25},

{"IF108", "Rekayasa Perangkat Lunak", 3, 4, 20},

{"IF109", "Seminar Hasil", 2, 7, 2}, // Kuota kecil (2) untuk
uji kuota penuh

{"IF110", "Skripsi", 6, 8, 1}, // Kuota sangat kecil (1)
untuk uji kuota penuh

}

for _, c := range coursesData {

    _, err := pool.Exec(ctx, `

        INSERT INTO courses (kode_mk, nama_mk, sks, semester,
kuota)

        VALUES ($1, $2, $3, $4, $5)

        ON CONFLICT (kode_mk) DO UPDATE SET

            nama_mk = EXCLUDED.nama_mk,

            sks = EXCLUDED.sks,

            semester = EXCLUDED.semester,

            kuota = EXCLUDED.kuota

    `, c.KodeMK, c>NamaMK, c.SKS, c.Semester, c.Kuota)

    if err != nil {

        log.Fatalf("Gagal seed course %s: %v", c.KodeMK, err)

    }

}

}

```

Akun yang dipakai:

- Admin

```
{ "email": "admin@siakad.test", "password": "admin12345"}
```

- Mahasiswa

```
{ "email": "mhsbaru01@siakad.test", "password": "202401000001"}
```

```
{ "email": "mhsbaru03@siakad.test", "password": "202401000003"}
```

```
{ "email": "mhsbaru05@siakad.test", "password": "202401000005"}
```

3.3 Helper Response

```
package helper

import (
    "api-students/app/model"
    "github.com/gofiber/fiber/v2"
)

type WebResponse struct {
    Success bool        `json:"success"`
    Message string       `json:"message"`
    Data    any           `json:"data,omitempty"`
    Meta    *model.Meta `json:"meta,omitempty"`
}

func Ok(c *fiber.Ctx, status int, message string, data any) error {
    return c.Status(status).JSON(WebResponse{
        Success: true, Message: message, Data: data,
    })
}
```

```
func OkList(c *fiber.Ctx, message string, data any, meta *model.Meta)
error {

    return c.Status(fiber.StatusOK).JSON(WebResponse{

        Success: true, Message: message, Data: data, Meta: meta,

    })

}
```

```
func Created(c *fiber.Ctx, message string, data any, location string)
error {

    if location != "" {

        c.Set("Location", location)

    }

    return c.Status(fiber.StatusCreated).JSON(WebResponse{

        Success: true, Message: message, Data: data,

    })

}
```

```
func NoContent(c *fiber.Ctx) error {

    return c.SendStatus(fiber.StatusNoContent)

}
```

```
func Success(c *fiber.Ctx, status int, message string, data any) error
{

    return c.Status(status).JSON(WebResponse{

        Success: true, Message: message, Data: data,
```

```
    })  
}
```

3.4 Students

```
type Student struct {  
    ID          int          `json:"id"`  
  
    UserID      int          `json:"user_id"`  
  
    Nim         string       `json:"nim"`  
  
    Nama        string       `json:"nama"`  
  
    Prodi       string       `json:"prodi"`  
  
    Angkatan    int          `json:"angkatan"`  
  
    IpkTerakhir float64      `json:"ipk_terakhir"`  
  
    DeletedAt   pgtype.Timestamptz `json:"-"`  
  
    CreatedAt   time.Time  
    `json:"created_at,omitempty"`  
}  
  
type StudentDetailResponse struct {  
  
    ID          int          `json:"id"`  
  
    UserID      int          `json:"user_id"`  
  
    Nim         string       `json:"nim"`  
  
    Nama        string       `json:"nama"`  
  
    Prodi       string       `json:"prodi"`  

```

```
    Angkatan      int           `json:"angkatan"`

    IpkTerakhir   float64       `json:"ipk_terakhir"`

    MataKuliah    []CourseItem `json:"mata_kuliah"`

    TotalSKS      int           `json:"total_sks"`

    BatasSKS      int           `json:"batas_sks"`

    CreatedAt     time.Time     `json:"created_at,omitempty"`
}
```

```
package repository

import (
    "context"

    "database/sql"

    "errors"

    "fmt"

    "time"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgconn"
    "github.com/jackc/pgx/v5/pgxpool"

    "api-students/app/model"
)
```

```

var (
    ErrNotFound = errors.New("data tidak ditemukan")

    ErrDuplicate = errors.New("data sudah terdaftar")
)

type StudentRepository interface {

    FindAll(ctx context.Context, q model.StudentListQuery)
    ([]model.Student, int, error)

    FindByID(ctx context.Context, id int) (model.Student, error)

    FindByUserID(ctx context.Context, userID int) (model.Student,
error)

    Create(ctx context.Context, s model.Student) (model.Student, error)

    CreateTx(ctx context.Context, tx pgx.Tx, s model.Student)
(model.Student, error)

    Update(ctx context.Context, s model.Student) (model.Student, error)

    SoftDelete(ctx context.Context, id int) error

    FindByIDWithCourses(ctx context.Context, id int)
(model.StudentDetailResponse, error)

    FindByIDWithPrestasi(ctx context.Context, id int)
(model.StudentWithPrestation, error)
}

type studentPostgresRepository struct {

    pool *pgxpool.Pool
}

func NewStudentRepository(pool *pgxpool.Pool) StudentRepository {

```

```

        return &studentPostgresRepository{pool: pool}
    }

var allowedSortColumns = map[string]string{

    "nama":          "nama ASC",

    "-ipk_terakhir": "ipk_terakhir DESC",

}

func (r *studentPostgresRepository) FindAll(

    ctx context.Context, q model.StudentListQuery,

) ([]model.Student, int, error) {

    where := " WHERE deleted_at IS NULL"

    args := []any{}

    if q.Prodi != "" {

        args = append(args, q.Prodi)

        where += fmt.Sprintf(" AND prodi = %d", len(args))

    }

    if q.Angkatan > 0 {

        args = append(args, q.Angkatan)

        where += fmt.Sprintf(" AND angkatan = %d", len(args))

    }

    if q.Search != "" {

        args = append(args, "%"+q.Search+"%")
    }

```

```

        where += fmt.Sprintf(" AND (nim ILIKE $%d OR nama ILIKE $%d)",
len(args), len(args))

    }

    var total int

    countQuery := "SELECT COUNT(*) FROM students" + where

    err := r.pool.QueryRow(ctx, countQuery, args...).Scan(&total)

    if err != nil {

        return nil, 0, fmt.Errorf("menghitung total student: %w", err)

    }

    orderBy := "ORDER BY id ASC"

    if sorted, ok := allowedSortColumns[q.Sort]; ok {

        orderBy = "ORDER BY " + sorted

    }

    sqlText := fmt.Sprintf(

        `SELECT id, user_id, nim, nama, prodi, angkatan, ipk_terakhir,
created_at

        FROM students%s

        %s

        LIMIT $%d OFFSET $%d`,

        where, orderBy, len(args)+1, len(args)+2,

    )

    args = append(args, q.PerPage, q.Offset())

```

```

rows, err := r.pool.Query(ctx, sqlText, args...)

if err != nil {

    return nil, 0, fmt.Errorf("mengambil daftar student: %w", err)

}

defer rows.Close()

list := []model.Student{}

for rows.Next() {

    var s model.Student

    if err := rows.Scan(&s.ID, &s.UserID, &s.Nim, &s>Nama,
&s.Prodi, &s.Angkatan, &s.IpkTerakhir, &s.CreatedAt); err != nil {

        return nil, 0, fmt.Errorf("membaca baris student: %w", err)

    }

    list = append(list, s)

}

if err := rows.Err(); err != nil {

    return nil, 0, fmt.Errorf("membaca hasil query: %w", err)

}

return list, total, nil
}

func (r *studentPostgresRepository) FindByID(ctx context.Context, id
int) (model.Student, error) {

    var s model.Student

    var deletedAt *time.Time

```

```

    err := r.pool.QueryRow(ctx,

        `SELECT id, user_id, nim, nama, prodi, angkatan, ipk_terakhir,
deleted_at, created_at

        FROM students WHERE id = $1 AND deleted_at IS NULL`, id,

    ).Scan(&s.ID, &s.UserID, &s.Nim, &s>Nama, &s.Prodi, &s.Angkatan,
&s.IpkTerakhir, &deletedAt, &s.CreatedAt)

    if err != nil {

        if errors.Is(err, pgx.ErrNoRows) {

            return model.Student{}, ErrNotFound

        }

        return model.Student{}, fmt.Errorf("mengambil student: %w",
err)

    }

    return s, nil

}

```

3.5 Courses

```

import "time"

type Course struct {

    ID      int    `json:"id"`

    KodeMK  string `json:"kode_mk"`

    NamaMK  string `json:"nama_mk"`

    SKS     int    `json:"sks"`

```

```

Semester int    `json:"semester"`

Kuota    int    `json:"kuota"`

Terisi   int    `json:"terisi"`

SisaKuota int    `json:"sisakuota"`

CreatedAt time.Time `json:"created_at,omitempty"`
}

type CourseQuery struct {

    Semester *int

    Search string

    Available bool
}

```

```

package repository

import (
    "context"
    "errors"
    "fmt"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"

    "api-students/app/model"
)

```

```

)

type CourseRepository interface {

    FindAll(ctx context.Context, q model.CourseQuery) ([]model.Course, error)

    FindByID(ctx context.Context, id int) (model.Course, error)

    FindByIDTxForUpdate(ctx context.Context, tx pgx.Tx, id int) (model.Course, error)

}

type coursePostgresRepository struct {

    pool *pgxpool.Pool

}

func NewCourseRepository(pool *pgxpool.Pool) CourseRepository {

    return &coursePostgresRepository{pool: pool}

}

func (r *coursePostgresRepository) FindAll(ctx context.Context, q model.CourseQuery) ([]model.Course, error) {

    where := " WHERE 1 = 1"

    args := []any{}

    if q.Semester != nil && *q.Semester > 0 {

        args = append(args, *q.Semester)

        where += fmt.Sprintf(" AND c.semester = %d", len(args))

    }

}

```

```

if q.Search != "" {

    args = append(args, "%"+q.Search+"%")

    where += fmt.Sprintf(" AND (c.kode_mk ILIKE $%d OR c.nama_mk ILIKE $%d)", len(args),
len(args))

}

if q.Available {

    where += " AND (c.kuota - COALESCE(e.terisi, 0)) > 0"

}

query := fmt.Sprintf(`

    SELECT c.id, c.kode_mk, c.nama_mk, c.sks, c.semester, c.kuota,

        COALESCE(e.terisi, 0)::int AS terisi,

        (c.kuota - COALESCE(e.terisi, 0))::int AS sisa_kuota

    FROM courses c

    LEFT JOIN (

        SELECT course_id, COUNT(*) AS terisi

        FROM enrollments

        GROUP BY course_id

    ) e ON e.course_id = c.id

    %s

    ORDER BY c.kode_mk ASC

`, where)

```

```

rows, err := r.pool.Query(ctx, query, args...)

if err != nil {

    return nil, fmt.Errorf("mengambil daftar mata kuliah: %w", err)

}

defer rows.Close()

courses := []model.Course{}

for rows.Next() {

    var c model.Course

    if err := rows.Scan(&c.ID, &c.KodeMK, &c>NamaMK, &c.SKS, &c.Semester, &c.Kuota, &c.Terisi,
&c.SisaKuota); err != nil {

        return nil, fmt.Errorf("membaca baris mata kuliah: %w", err)

    }

    courses = append(courses, c)

}

if err := rows.Err(); err != nil {

    return nil, fmt.Errorf("membaca hasil query mata kuliah: %w", err)

}

return courses, nil
}

func (r *coursePostgresRepository) FindByID(ctx context.Context, id int) (model.Course, error) {

    var c model.Course

    query := `

        SELECT c.id, c.kode_mk, c.nama_mk, c.sks, c.semester, c.kuota,

```

```

        COALESCE(e.terisi, 0)::int AS terisi,

        (c.kuota - COALESCE(e.terisi, 0))::int AS sisa_kuota

    FROM courses c

    LEFT JOIN (

        SELECT course_id, COUNT(*) AS terisi

        FROM enrollments

        WHERE course_id = $1

        GROUP BY course_id

    ) e ON e.course_id = c.id

    WHERE c.id = $1
    ,

err := r.pool.QueryRow(ctx, query, id).Scan(&c.ID, &c.KodeMK, &c>NamaMK, &c.SKS, &c.Semester,
&c.Kuota, &c.Terisi, &c.SisaKuota)

if err != nil {

    if errors.Is(err, pgx.ErrNoRows) {

        return model.Course{}, ErrNotFound

    }

    return model.Course{}, fmt.Errorf("mengambil mata kuliah: %w", err)

}

return c, nil
}

func (r *coursePostgresRepository) FindByIDTxForUpdate(ctx context.Context, tx pgx.Tx, id int)
(model.Course, error) {

    var c model.Course

    query := `

```

```

SELECT c.id, c.kode_mk, c.nama_mk, c.sks, c.semester, c.kuota,

        COALESCE(e.terisi, 0)::int AS terisi,

        (c.kuota - COALESCE(e.terisi, 0))::int AS sisa_kuota

FROM courses c

LEFT JOIN (

        SELECT course_id, COUNT(*) AS terisi

        FROM enrollments

        WHERE course_id = $1

        GROUP BY course_id

    ) e ON e.course_id = c.id

WHERE c.id = $1

FOR UPDATE OF c
,

err := tx.QueryRow(ctx, query, id).Scan(&c.ID, &c.KodeMK, &c>NamaMK, &c.SKS, &c.Semester,
&c.Kuota, &c.Terisi, &c.SisaKuota)

if err != nil {

    if errors.Is(err, pgx.ErrNoRows) {

        return model.Course{}, ErrNotFound

    }

    return model.Course{}, fmt.Errorf("mengambil mata kuliah dengan row lock: %w", err)

}

return c, nil
}

package service

import (

```

```
"strings"
```

```
"github.com/gofiber/fiber/v2"
```

```
"api-students/app/model"
```

```
"api-students/app/repository"
```

```
"api-students/helper"
```

```
)
```

```
type CourseService struct {
```

```
    repo repository.CourseRepository
```

```
}
```

```
func NewCourseService(repo repository.CourseRepository) *CourseService {
```

```
    return &CourseService{repo: repo}
```

```
}
```

```
func (s *CourseService) List(c *fiber.Ctx) error {
```

```
    ctx, cancel := helper.ReqCtx(c)
```

```
    defer cancel()
```

```
    q := model.CourseQuery{
```

```
        Search: strings.TrimSpace(c.Query("search")),
```

```
        Available: c.QueryBool("available", false),
```

```
    }
```

```

if semesterStr := c.Query("semester"); semesterStr != "" {

    semester := c.QueryInt("semester", 0)

    if semester > 0 {

        q.Semester = &semester

    }

}

courses, err := s.repo.FindAll(ctx, q)

if err != nil {

    return helper.Internal(err)

}

return helper.Ok(c, fiber.StatusOK, "Daftar mata kuliah berhasil diambil", courses)
}

```

3.6 Enrollments

```

package model

import "time"

type Enrollment struct {

    ID          int          `json:"id"`

    StudentID   int          `json:"student_id"`

    CourseID    int          `json:"course_id"`

```

```

    TahunAkademik string `json:"tahun_akademik"`

    CreatedAt      time.Time `json:"created_at"`
}

type CreateEnrollmentRequest struct {

    CourseID      int    `json:"course_id" validate:"required,min=1"`

    TahunAkademik string `json:"tahun_akademik" validate:"required"`
}

```

```

package repository

import (
    "context"
    "errors"
    "fmt"

    "github.com/jackc/pgx/v5"
    "github.com/jackc/pgx/v5/pgxpool"

    "api-students/app/model"
)

type EnrollmentRepository interface {
    CreateTx(ctx context.Context, tx pgx.Tx, e model.Enrollment)
(model.Enrollment, error)
    CountByCourseTx(ctx context.Context, tx pgx.Tx, courseID int) (int,
error)
    GetTotalSKSTx(ctx context.Context, tx pgx.Tx, studentID int,
tahunAkademik string) (int, error)
    ExistsTx(ctx context.Context, tx pgx.Tx, studentID, courseID int,
tahunAkademik string) (bool, error)
    FindByID(ctx context.Context, id int) (model.Enrollment, error)
    Delete(ctx context.Context, id int) error
}

type enrollmentPostgresRepository struct {
    pool *pgxpool.Pool
}

```

```

}

func NewEnrollmentRepository(pool *pgxpool.Pool) EnrollmentRepository {
    return &enrollmentPostgresRepository{pool: pool}
}

func (r *enrollmentPostgresRepository) CreateTx(ctx context.Context, tx
pgx.Tx, e model.Enrollment) (model.Enrollment, error) {
    err := tx.QueryRow(ctx,
        `INSERT INTO enrollments (student_id, course_id,
tahun_akademik)
        VALUES ($1, $2, $3)
        RETURNING id, created_at`,
        e.StudentID, e.CourseID, e.TahunAkademik,
    ).Scan(&e.ID, &e.CreatedAt)

    if err != nil {
        if isUniqueViolation(err) {
            return model.Enrollment{}, ErrDuplicate
        }
        return model.Enrollment{}, fmt.Errorf("menyimpan enrollment:
%w", err)
    }
    return e, nil
}

func (r *enrollmentPostgresRepository) CountByCourseTx(ctx
context.Context, tx pgx.Tx, courseID int) (int, error) {
    var count int
    err := tx.QueryRow(ctx,
        `SELECT COUNT(*) FROM enrollments WHERE course_id = $1`,
courseID,
    ).Scan(&count)
    if err != nil {
        return 0, fmt.Errorf("menghitung enrollment course: %w", err)
    }
    return count, nil
}

func (r *enrollmentPostgresRepository) GetTotalSKSTx(ctx
context.Context, tx pgx.Tx, studentID int, tahunAkademik string) (int,
error) {
    var total int

```

```

err := tx.QueryRow(ctx,
    `SELECT COALESCE(SUM(c.sks), 0)
    FROM enrollments e
    JOIN courses c ON c.id = e.course_id
    WHERE e.student_id = $1 AND e.tahun_akademik = $2`,
    studentID, tahunAkademik,
).Scan(&total)

if err != nil {
    return 0, fmt.Errorf("menghitung total SKS student: %w", err)
}
return total, nil
}

func (r *enrollmentPostgresRepository) ExistsTx(ctx context.Context, tx
pgx.Tx, studentID, courseID int, tahunAkademik string) (bool, error) {
    var exists bool
    err := tx.QueryRow(ctx,
        `SELECT EXISTS(
            SELECT 1 FROM enrollments
            WHERE student_id = $1 AND course_id = $2 AND tahun_akademik
= $3
        )`, studentID, courseID, tahunAkademik,
    ).Scan(&exists)
    if err != nil {
        return false, fmt.Errorf("memeriksa keberadaan enrollment: %w",
err)
    }
    return exists, nil
}

func (r *enrollmentPostgresRepository) FindByID(ctx context.Context, id
int) (model.Enrollment, error) {
    var e model.Enrollment
    err := r.pool.QueryRow(ctx,
        `SELECT id, student_id, course_id, tahun_akademik, created_at
        FROM enrollments WHERE id = $1`, id,
    ).Scan(&e.ID, &e.StudentID, &e.CourseID, &e.TahunAkademik,
&e.CreatedAt)

    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) {
            return model.Enrollment{}, ErrNotFound

```

```

    }
    return model.Enrollment{}, fmt.Errorf("mengambil enrollment: %w", err)
}

return e, nil
}

func (r *enrollmentPostgresRepository) Delete(ctx context.Context, id
int) error {
    cmd, err := r.pool.Exec(ctx, `DELETE FROM enrollments WHERE id =
$1`, id)
    if err != nil {
        return fmt.Errorf("menghapus enrollment: %w", err)
    }
    if cmd.RowsAffected() == 0 {
        return ErrNotFound
    }
    return nil
}

```

3.7 Route

```

package route

import (
    "context"
    "time"

    "github.com/gofiber/fiber/v2"
    "github.com/jackc/pgx/v5/pgxpool"

    "api-students/app/service"
    "api-students/helper"
    "api-students/middleware"
)

type Dependencies struct {
    Pool          *pgxpool.Pool
    JWT           *helper.JWTManager
    AuthService   *service.AuthService
    StudentService *service.StudentService
    CourseService *service.CourseService
    EnrollmentService *service.EnrollmentService
}

```

```

}

func Register(app *fiber.App, deps Dependencies) {
    api := app.Group("/api/v1")

    api.Get("/health", healthCheck(deps.Pool))

    // 1. Auth routes
    auth := api.Group("/auth")
    auth.Post("/login", middleware.RequireJSON,
middleware.LoginRateLimiter(), deps.AuthService.Login)
    auth.Get("/me", middleware.RequireAuth(deps.JWT),
deps.AuthService.Me)

    // 2. Students routes
    students := api.Group("/students",
middleware.RequireAuth(deps.JWT))
    students.Get("/", middleware.RequireRole("admin"),
deps.StudentService.List)
    students.Post("/", middleware.RequireJSON,
middleware.RequireRole("admin"), deps.StudentService.Create)
    students.Get("/:id", deps.StudentService.Get)
    students.Put("/:id", middleware.RequireJSON,
middleware.RequireRole("admin"), deps.StudentService.Update)
    students.Delete("/:id", middleware.RequireRole("admin"),
deps.StudentService.Delete)

    // Legacy prestasi route
    students.Get("/:id/prestasi",
deps.StudentService.GetWithPrestation)

    // 3. Courses routes
    courses := api.Group("/courses", middleware.RequireAuth(deps.JWT))
    courses.Get("/", deps.CourseService.List)

    // 4. Enrollments routes
    enrollments := api.Group("/enrollments",
middleware.RequireAuth(deps.JWT))
    enrollments.Post("/", middleware.RequireJSON,
middleware.RequireRole("mahasiswa"), deps.EnrollmentService.Create)
    enrollments.Delete("/:id", middleware.RequireRole("mahasiswa"),
deps.EnrollmentService.Delete)
}

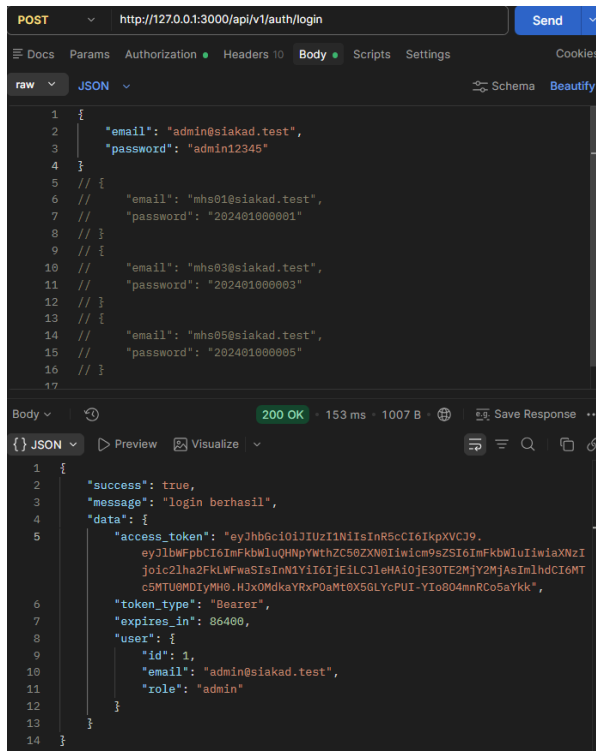
```

```
func healthCheck(pool *pgxpool.Pool) fiber.Handler {
    return func(c *fiber.Ctx) error {
        ctx, cancel := context.WithTimeout(c.UserContext(),
2*time.Second)
        defer cancel()

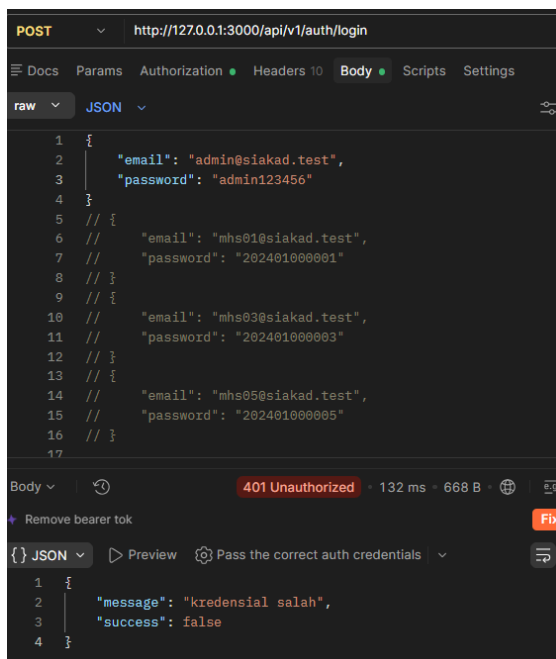
        if err := pool.Ping(ctx); err != nil {
            return helper.ServiceUnavailable("database tidak dapat
dihubungi")
        }
        return helper.Success(c, fiber.StatusOK, "server dan database
berjalan", nil)
    }
}
```

BAB IV PENGUJIAN

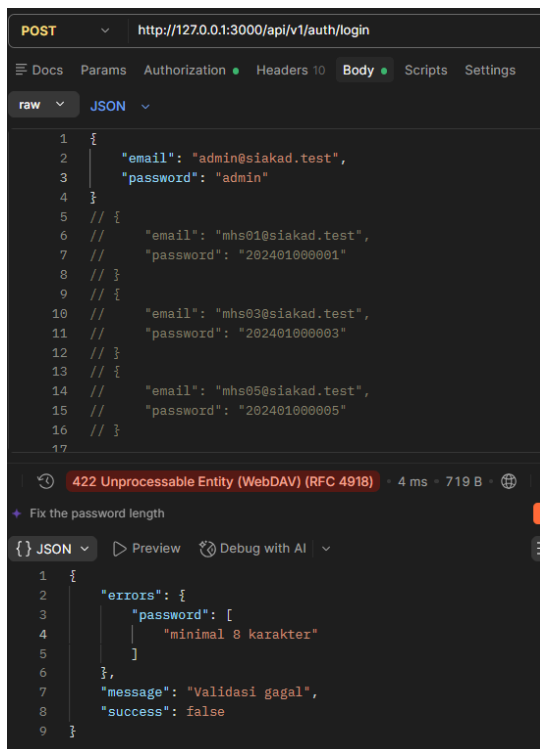
1. POST /auth/login, Kredensial benar, 200, berisi access_token



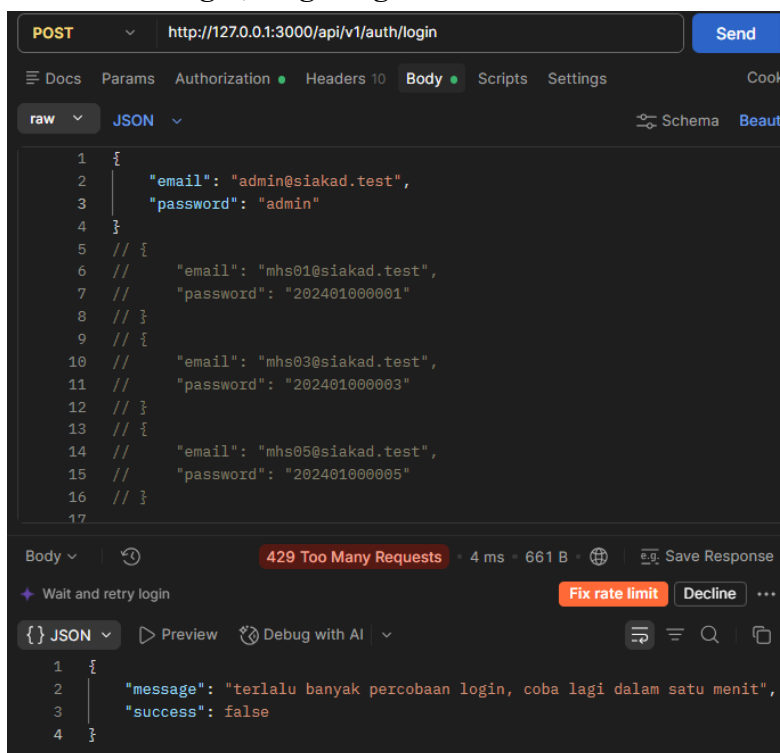
2. POST /auth/login, Password salah, 401



3. POST /auth/login, Email tidak valid atau password kurang dari 8 karakter, 422



4. POST /auth/login, Gagal login lebih dari 5 kali dalam satu menit, 429



5. GET /auth/me, Token valid (admin dan mahasiswa), 200, mahasiswa menyertakan data students

- Admin

```
GET http://127.0.0.1:3000/api/v1/auth/me

{
  "email": "admin@siakad.test",
  "password": "admin12345"
}

200 OK - 3 ms - 701 B

{
  "success": true,
  "message": "profil berhasil diambil",
  "data": {
    "id": 1,
    "email": "admin@siakad.test",
    "role": "admin",
    "created_at": "2026-10-09T16:42:17.707428+07:00"
  }
}
```

- Mahasiswa

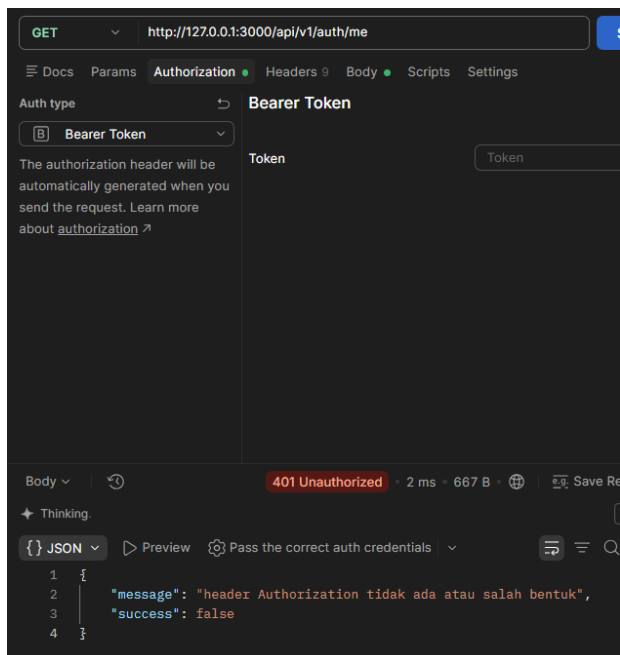
```
GET http://127.0.0.1:3000/api/v1/auth/me

// {
//   "email": "admin@siakad.test",
//   "password": "admin12345"
// }
{
  "email": "mhs01@siakad.test",
  "password": "202401000001"
}
// {
//   "email": "mhs03@siakad.test",
//   "password": "202401000003"
// }
// {
//   "email": "mhs05@siakad.test",
//   "password": "202401000005"
// }

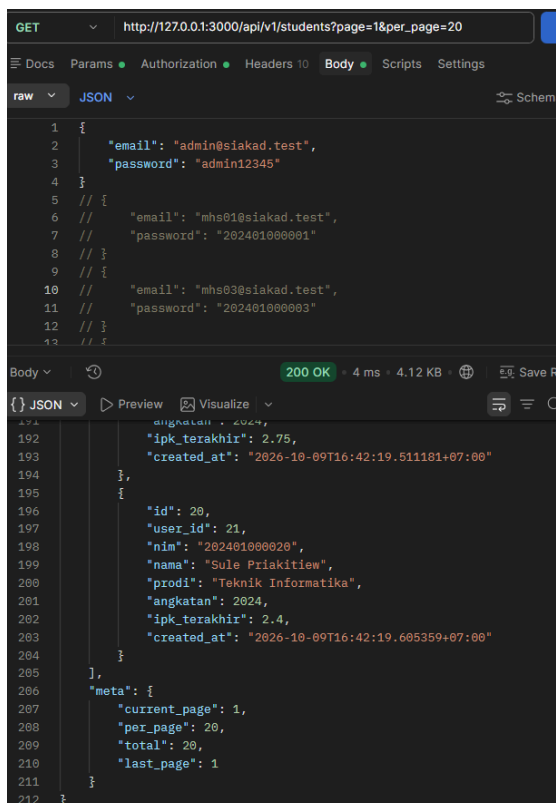
200 OK - 12 ms - 831 B

{
  "success": true,
  "message": "profil berhasil diambil",
  "data": {
    "id": 2,
    "email": "mhs01@siakad.test",
    "role": "mahasiswa",
    "created_at": "2026-10-09T16:42:17.82693+07:00",
    "student": {
      "id": 1,
      "nim": "202401000001",
      "nama": "Budi Santoso",
      "prodi": "Teknik Informatika",
      "angkatan": 2024,
      "ipk_terakhir": 3.75
    }
  }
}
```

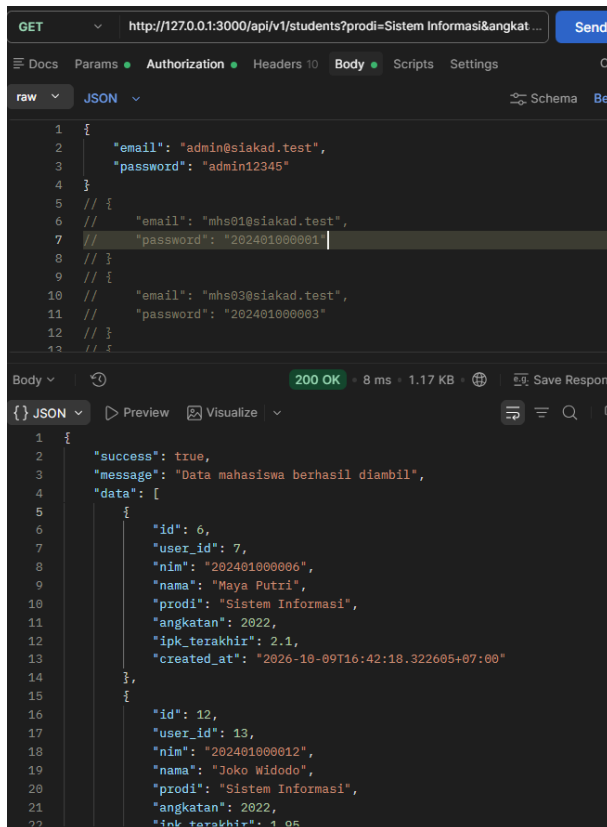
6. GET /auth/me, Tanpa token atau token kedaluwarsa, 401



7. GET /students, Admin dengan page, per_page, filter, search, dan sort, 200 dengan data dan meta - page & per_page



- Filter



```
GET http://127.0.0.1:3000/api/v1/students?prodi=Sistem Informasi&angkatan=2022 Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Schema

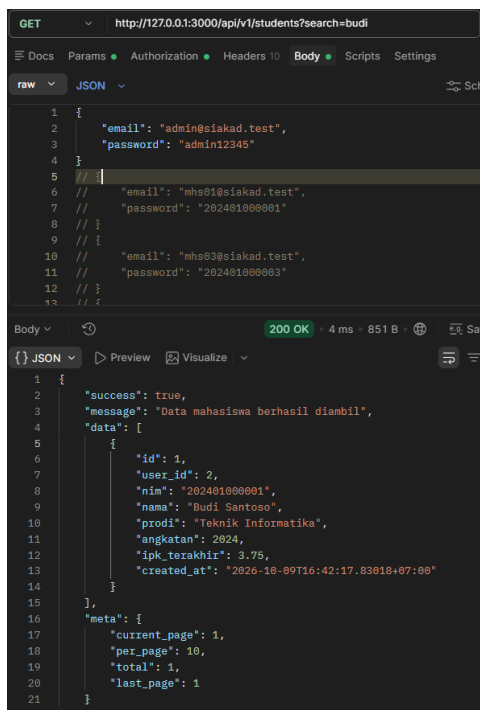
1 {
2   "email": "admin@siakad.test",
3   "password": "admin12345"
4 }
5 // {
6 //   "email": "mhs01@siakad.test",
7 //   "password": "202401000001"
8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 // {

Body 200 OK 8 ms 1.17 KB Save Response

JSON Preview Visualize

1 {
2   "success": true,
3   "message": "Data mahasiswa berhasil diambil",
4   "data": [
5     {
6       "id": 6,
7       "user_id": 7,
8       "nim": "202401000006",
9       "nama": "Maya Putri",
10      "prodi": "Sistem Informasi",
11      "angkatan": 2022,
12      "ipk_terakhir": 2.1,
13      "created_at": "2026-10-09T16:42:18.322605+07:00"
14    },
15    {
16      "id": 12,
17      "user_id": 13,
18      "nim": "202401000012",
19      "nama": "Joko Widodo",
20      "prodi": "Sistem Informasi",
21      "angkatan": 2022,
22      "ipk_terakhir": 4.05
23    }
24  ]
25 }
```

- Search



```
GET http://127.0.0.1:3000/api/v1/students?search=budi

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Schema

1 {
2   "email": "admin@siakad.test",
3   "password": "admin12345"
4 }
5 // {
6 //   "email": "mhs01@siakad.test",
7 //   "password": "202401000001"
8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 // {

Body 200 OK 4 ms 851 B Save Response

JSON Preview Visualize

1 {
2   "success": true,
3   "message": "Data mahasiswa berhasil diambil",
4   "data": [
5     {
6       "id": 1,
7       "user_id": 2,
8       "nim": "202401000001",
9       "nama": "Budi Santoso",
10      "prodi": "Teknik Informatika",
11      "angkatan": 2024,
12      "ipk_terakhir": 3.75,
13      "created_at": "2026-10-09T16:42:17.83018+07:00"
14    }
15  ],
16  "meta": {
17    "current_page": 1,
18    "per_page": 10,
19    "total": 1,
20    "last_page": 1
21  }
22 }
23 }
```

- Sort

The screenshot displays a REST client interface with a GET request to `http://127.0.0.1:3000/api/v1/students?sort=-ipk_terakhir`. The response is a 200 OK status with a 76 ms response time and 2.38 KB of data. The response body is a JSON object containing a success message and a list of student data sorted by their final grade (ipk_terakhir) in descending order.

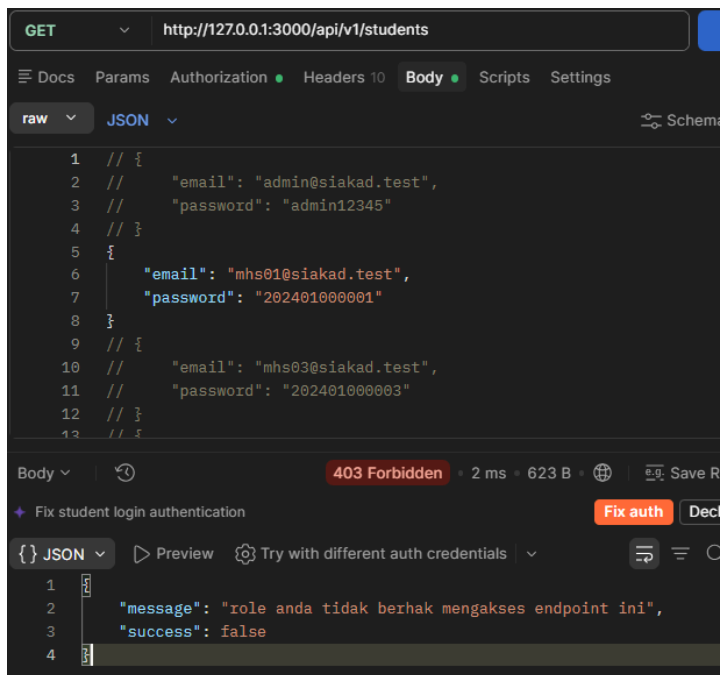
```
1 {
2   "email": "admin@siakad.test",
3   "password": "admin12345"
4 }
5 // {
6 //   "email": "mhs01@siakad.test",
7 //   "password": "202401000001"
8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 // }
```

Body ▾ 200 OK • 76 ms • 2.38 KB • Save Response

{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "success": true,
3   "message": "Data mahasiswa berhasil diambil",
4   "data": [
5     {
6       "id": 7,
7       "user_id": 8,
8       "nim": "202401000007",
9       "nama": "Eko Prasetyo",
10      "prodi": "Teknologi Informasi",
11      "angkatan": 2024,
12      "ipk_terakhir": 3.9,
13      "created_at": "2026-10-09T16:42:18.412006+07:00"
14    },
15    {
16      "id": 18,
17      "user_id": 19,
18      "nim": "202401000018",
19      "nama": "Qory Sandioriva",
20      "prodi": "Sistem Informasi",
21      "angkatan": 2022,
22      "ipk_terakhir": 3.85
```

8. GET /students, Login sebagai mahasiswa, 403



```
GET http://127.0.0.1:3000/api/v1/students

raw JSON

1 // {
2 //   "email": "admin@siakad.test",
3 //   "password": "admin12345"
4 // }
5 {
6   "email": "mhs01@siakad.test",
7   "password": "202401000001"
8 }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 // }
```

Body 403 Forbidden • 2 ms • 623 B

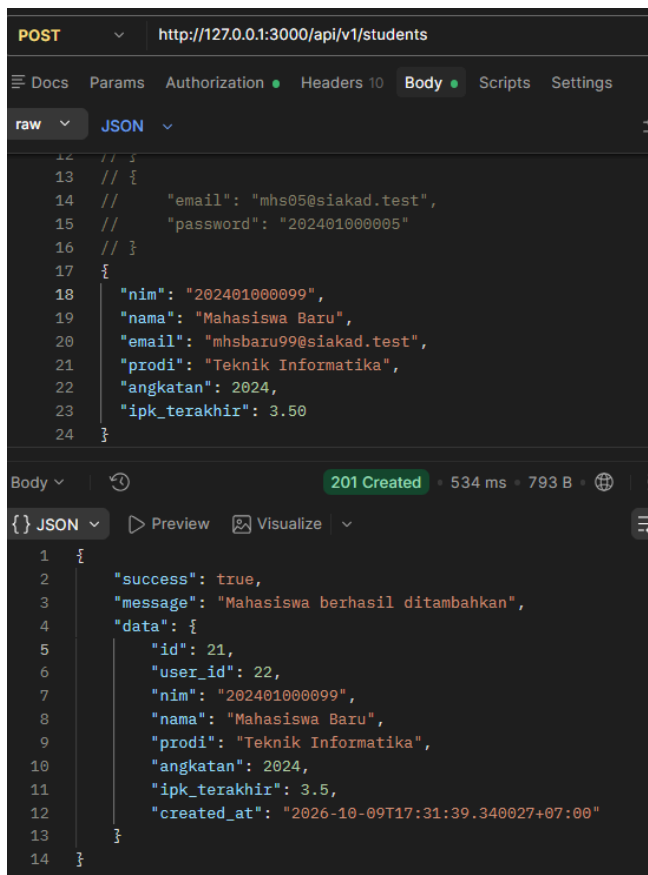
Fix student login authentication

Fix auth

JSON

```
1 {
2   "message": "role anda tidak berhak mengakses endpoint ini",
3   "success": false
4 }
```

9. POST /students, Data valid, 201, user dan student terbentuk



```
POST http://127.0.0.1:3000/api/v1/students

raw JSON

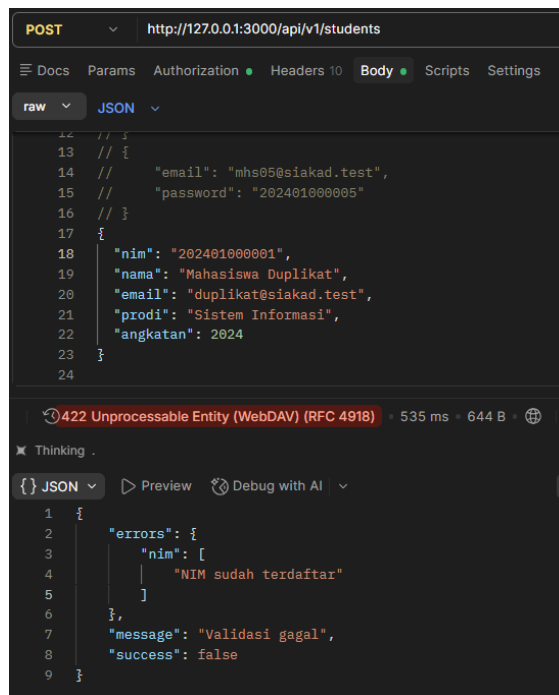
12 // {
13 //   "email": "mhs05@siakad.test",
14 //   "password": "202401000005"
15 // }
16 // {
17 {
18   "nim": "202401000099",
19   "nama": "Mahasiswa Baru",
20   "email": "mhsbaru99@siakad.test",
21   "prodi": "Teknik Informatika",
22   "angkatan": 2024,
23   "ipk_terakhir": 3.50
24 }
```

Body 201 Created • 534 ms • 793 B

JSON

```
1 {
2   "success": true,
3   "message": "Mahasiswa berhasil ditambahkan",
4   "data": {
5     "id": 21,
6     "user_id": 22,
7     "nim": "202401000099",
8     "nama": "Mahasiswa Baru",
9     "prodi": "Teknik Informatika",
10    "angkatan": 2024,
11    "ipk_terakhir": 3.5,
12    "created_at": "2026-10-09T17:31:39.340027+07:00"
13  }
14 }
```

10. POST /students, NIM atau email duplikat, 422



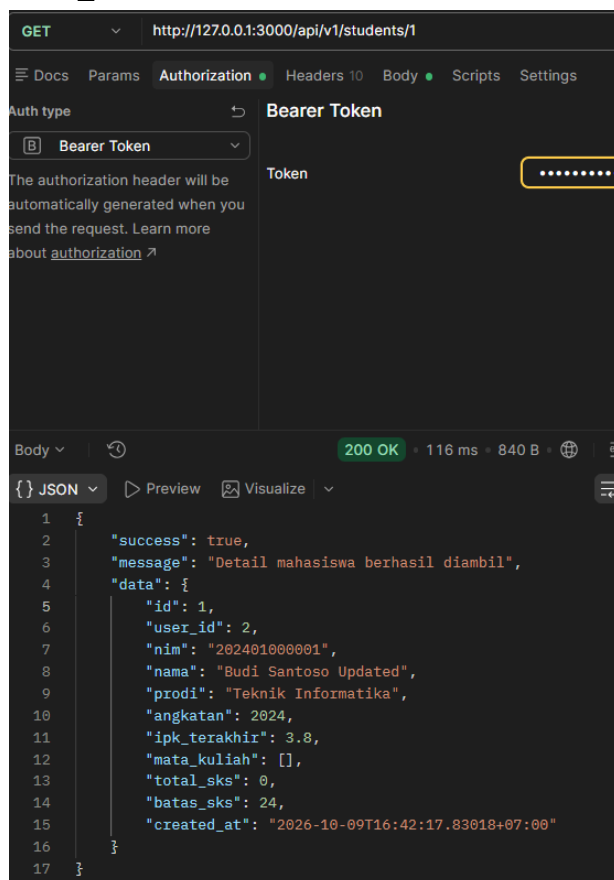
```
POST http://127.0.0.1:3000/api/v1/students

{
  "nim": "202401000001",
  "nama": "Mahasiswa Duplikat",
  "email": "duplikat@siakad.test",
  "prodi": "Sistem Informasi",
  "angkatan": 2024
}
```

422 Unprocessable Entity (WebDAV) (RFC 4918) • 535 ms • 644 B

```
{
  "errors": {
    "nim": [
      "NIM sudah terdaftar"
    ]
  },
  "message": "Validasi gagal",
  "success": false
}
```

11. GET /students/{id}, Mahasiswa melihat data sendiri, 200 dengan total_sks dan batas_sks



```
GET http://127.0.0.1:3000/api/v1/students/1
```

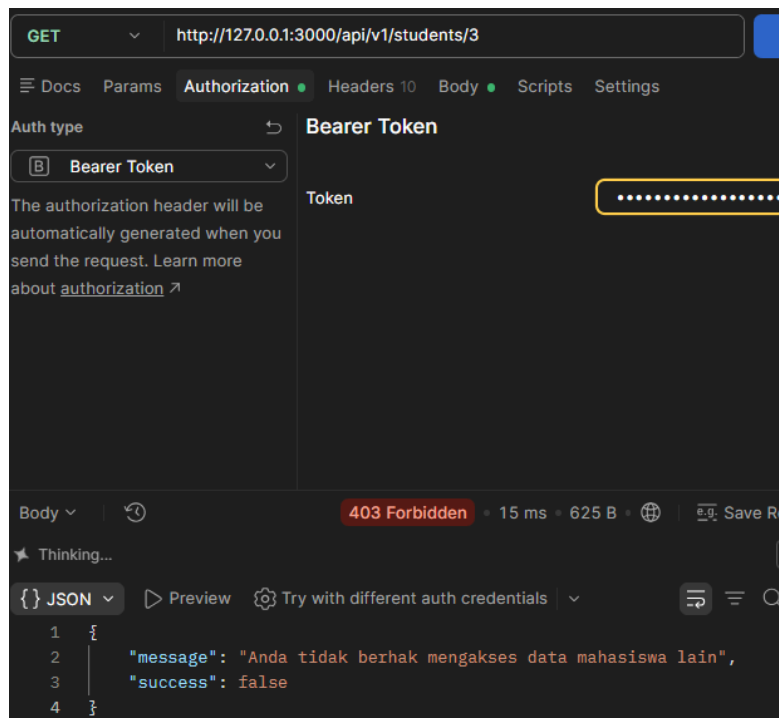
Auth type: Bearer Token

Token:

200 OK • 116 ms • 840 B

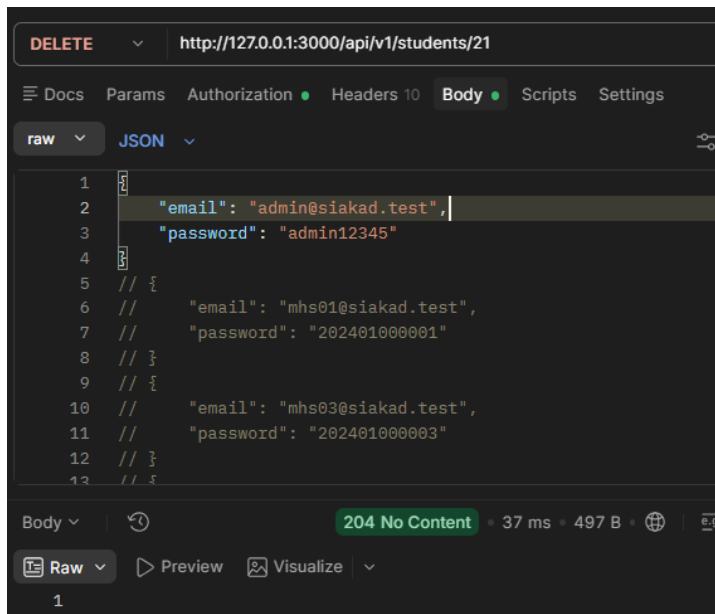
```
{
  "success": true,
  "message": "Detail mahasiswa berhasil diambil",
  "data": {
    "id": 1,
    "user_id": 2,
    "nim": "202401000001",
    "nama": "Budi Santoso Updated",
    "prodi": "Teknik Informatika",
    "angkatan": 2024,
    "ipk_terakhir": 3.8,
    "mata_kuliah": [],
    "total_sks": 0,
    "batas_sks": 24,
    "created_at": "2026-10-09T16:42:17.83018+07:00"
  }
}
```

12. GET /students/{id}, Mahasiswa melihat data mahasiswa lain, 403

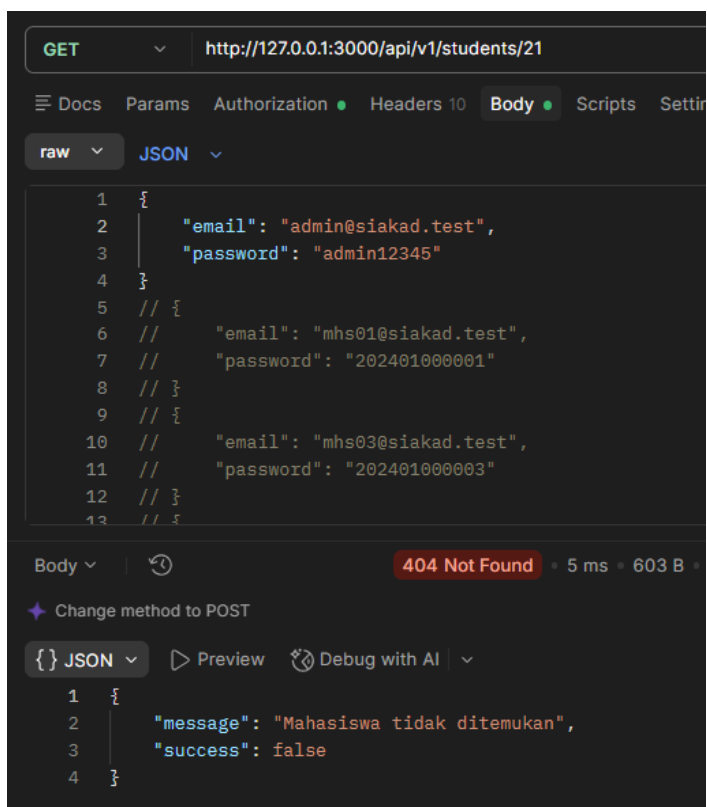


13. GET /students/{id}, ID tidak ada atau sudah di-soft delete, 404

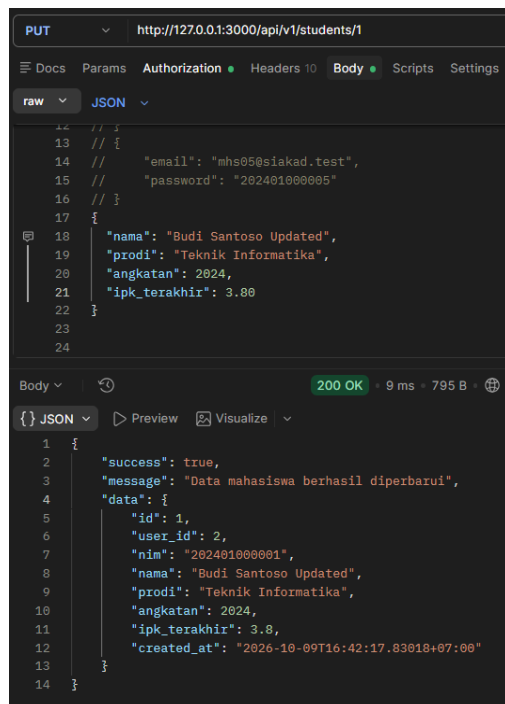
- Sebelum



- Sesudah



14. PUT /students/{id}, Admin memperbarui data valid, 200



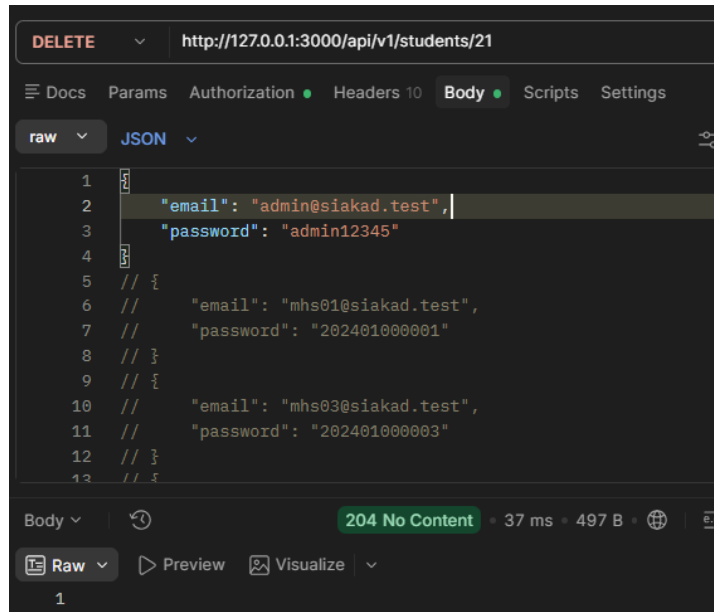
The screenshot displays a REST client interface with the following details:

- Method:** PUT
- URL:** http://127.0.0.1:3000/api/v1/students/1
- Body (Request):** A JSON object with the following structure:

```
{  "email": "mhs05@siakad.test",  "password": "202401000005",  "nama": "Budi Santoso Updated",  "prodi": "Teknik Informatika",  "angkatan": 2024,  "ipk_terakhir": 3.80}
```
- Status:** 200 OK
- Response Body (JSON):**

```
{  "success": true,  "message": "Data mahasiswa berhasil diperbarui",  "data": {    "id": 1,    "user_id": 2,    "nim": "202401000001",    "nama": "Budi Santoso Updated",    "prodi": "Teknik Informatika",    "angkatan": 2024,    "ipk_terakhir": 3.8,    "created_at": "2026-10-09T16:42:17.83818+07:00"  }}
```

15. DELETE /students/{id}, Admin menghapus mahasiswa, 204, tidak muncul di daftar



```
DELETE http://127.0.0.1:3000/api/v1/students/21
```

Docs Params Authorization Headers 10 Body Scripts Settings

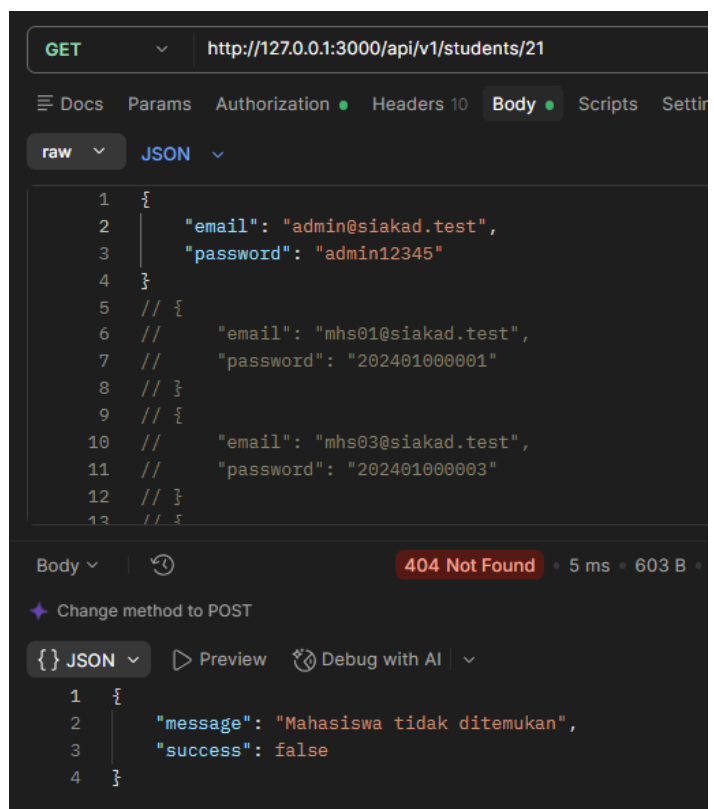
raw JSON

```
1 {
2   "email": "admin@siakad.test",
3   "password": "admin12345"
4 }
5 // {
6 //   "email": "mhs01@siakad.test",
7 //   "password": "202401000001"
8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 // }
```

Body 204 No Content 37 ms 497 B

Raw Preview Visualize

1



```
GET http://127.0.0.1:3000/api/v1/students/21
```

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON

```
1 {
2   "email": "admin@siakad.test",
3   "password": "admin12345"
4 }
5 // {
6 //   "email": "mhs01@siakad.test",
7 //   "password": "202401000001"
8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 // }
```

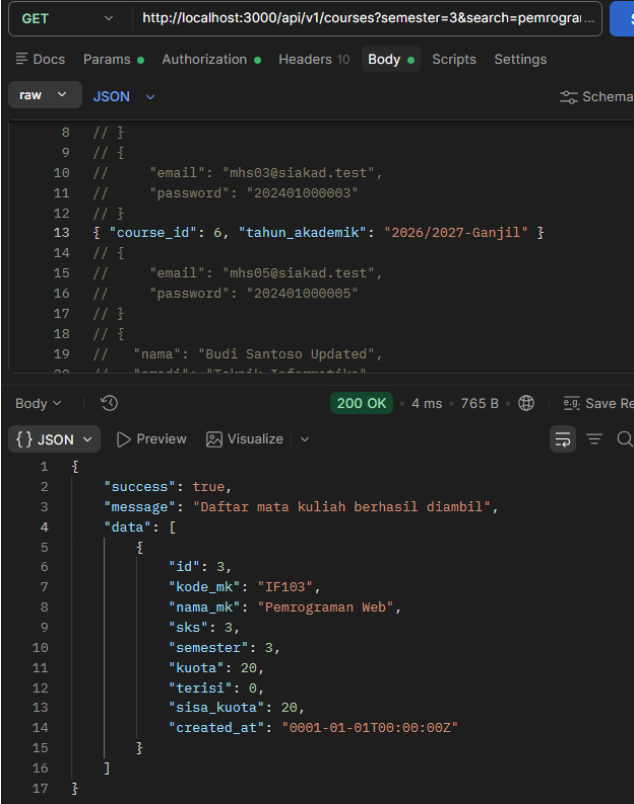
Body 404 Not Found 5 ms 603 B

Change method to POST

{ JSON Preview Debug with AI

```
1 {
2   "message": "Mahasiswa tidak ditemukan",
3   "success": false
4 }
```

16. GET /courses, Filter semester, search, dan available=true, 200 dengan terisi dan sisa_kuota



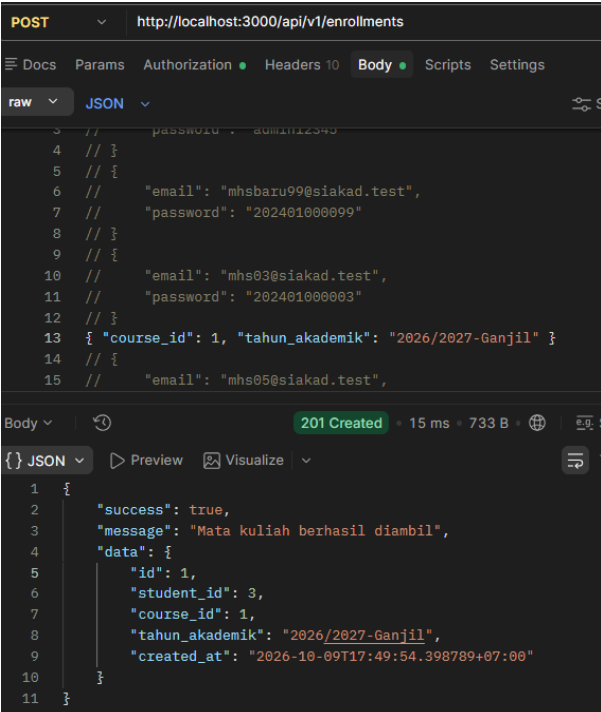
```
GET http://localhost:3000/api/v1/courses?semester=3&search=pemrograman...

Body
raw JSON
Schema

8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 { "course_id": 6, "tahun_akademik": "2026/2027-Ganjil" }
14 // {
15 //   "email": "mhs05@siakad.test",
16 //   "password": "202401000005"
17 // }
18 // {
19 //   "nama": "Budi Santoso Updated",
20 // }

Body 200 OK 4 ms 765 B
JSON Preview Visualize
1 {
2   "success": true,
3   "message": "Daftar mata kuliah berhasil diambil",
4   "data": [
5     {
6       "id": 3,
7       "kode_mk": "IF103",
8       "nama_mk": "Pemrograman Web",
9       "sks": 3,
10      "semester": 3,
11      "kuota": 20,
12      "terisi": 0,
13      "sisa_kuota": 20,
14      "created_at": "0001-01-01T00:00:00Z"
15    }
16  ]
17 }
```

17. POST /enrollments, Mengambil mata kuliah yang memenuhi syarat, 201



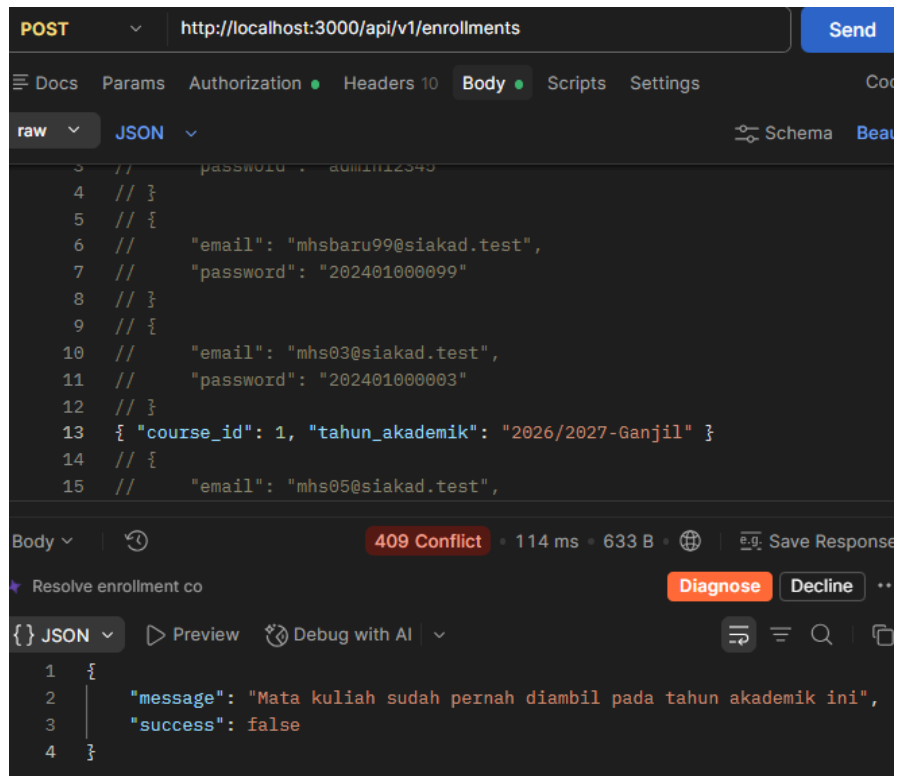
```
POST http://localhost:3000/api/v1/enrollments

Body
raw JSON
Schema

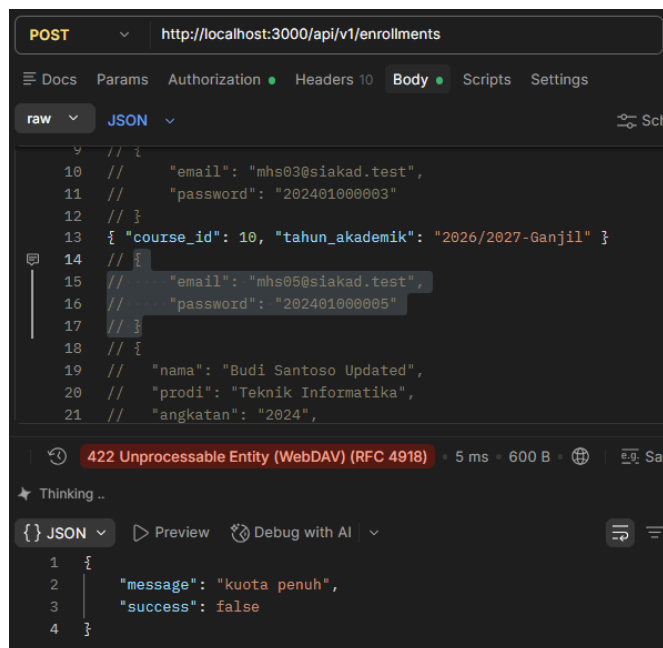
3 //   "password": "admin12345"
4 // }
5 // {
6 //   "email": "mhsbaru99@siakad.test",
7 //   "password": "202401000099"
8 // }
9 // {
10 //   "email": "mhs03@siakad.test",
11 //   "password": "202401000003"
12 // }
13 { "course_id": 1, "tahun_akademik": "2026/2027-Ganjil" }
14 // {
15 //   "email": "mhs05@siakad.test",

Body 201 Created 15 ms 733 B
JSON Preview Visualize
1 {
2   "success": true,
3   "message": "Mata kuliah berhasil diambil",
4   "data": {
5     "id": 1,
6     "student_id": 3,
7     "course_id": 1,
8     "tahun_akademik": "2026/2027-Ganjil",
9     "created_at": "2026-10-09T17:49:54.398789+07:00"
10   }
11 }
```

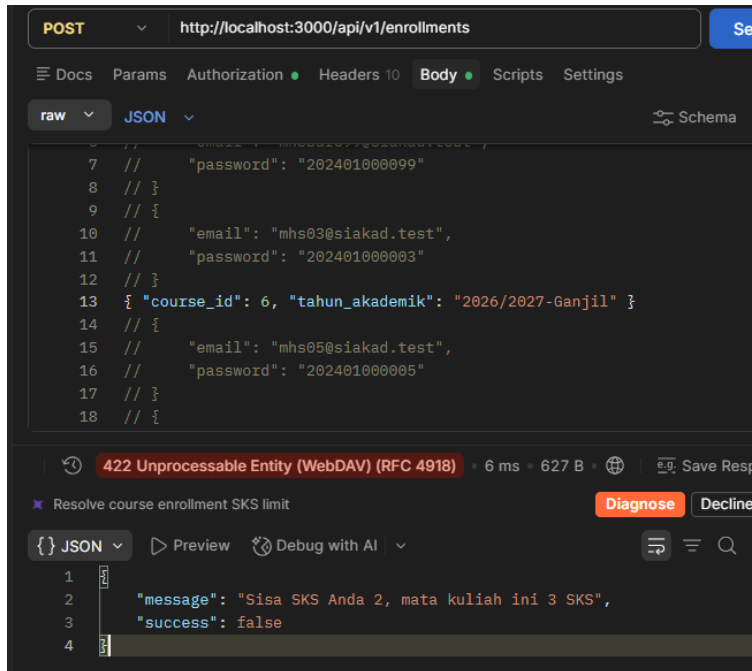
18. POST /enrollments, Mengambil mata kuliah yang sama pada tahun akademik yang sama, 409



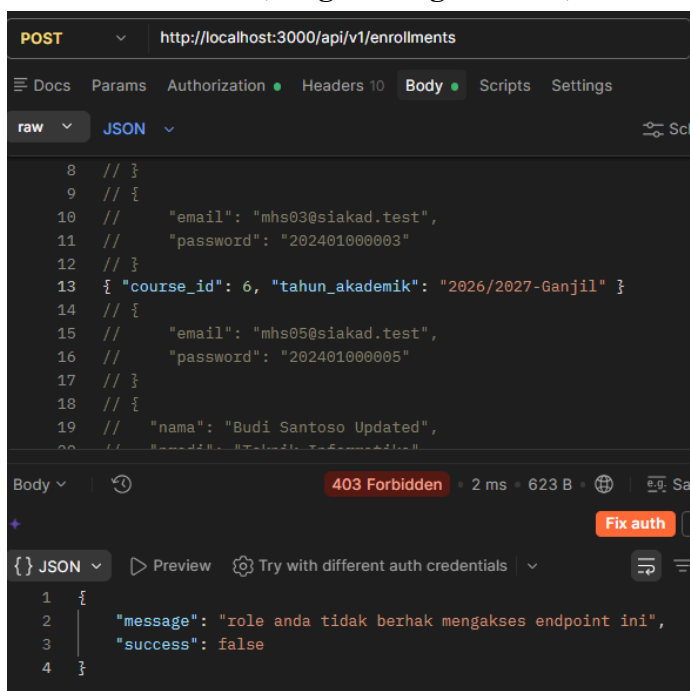
19. POST /enrollments, Mata kuliah berkuota penuh, 422



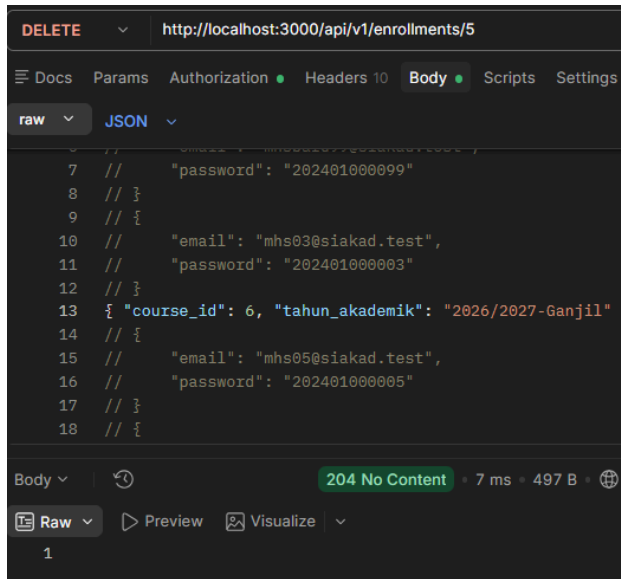
20. POST /enrollments, Total SKS melebihi batas (IPK $\geq 3,00$; 2,50 sampai 2,99; < 2,50), 422 dengan pesan sisa SKS



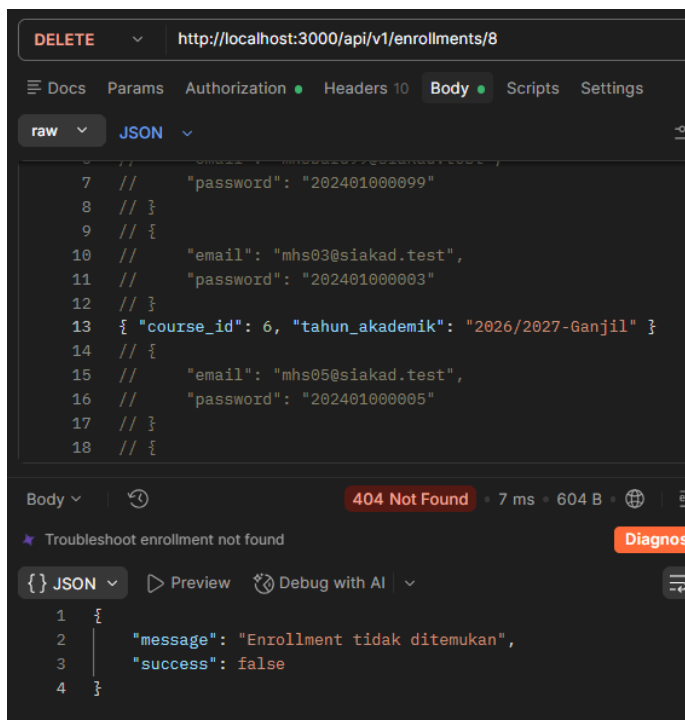
21. POST /enrollments, Login sebagai admin, 403



22. DELETE /enrollments/{id}, Membatalkan enrollment milik sendiri, 204, sisa kuota bertambah



23. DELETE /enrollments/{id}, ID tidak ditemukan, 404



LAMPIRAN

Link Github : <https://github.com/fahmi-blip/siakad.git>